1. Explain the features of Java. Why is Java platform-independent?

1. Definition

Java is a high-level, class-based, object-oriented programming language. It is used to develop desktop applications, web applications, enterprise software, and many other programs.

Java was developed by James Gosling and his team at Sun Microsystems. It was officially released in 1995.

2. Meaning

Java allows programmers to write instructions that a computer can execute. It provides features such as classes, objects, exception handling, automatic memory management, and platform independence.

For example, a Java program can calculate student marks, manage bank accounts, or display information on a screen.

3. Special features of java:

1. Simple

Java has clear syntax and removes several complex features found in C++, such as direct pointer manipulation. This makes it easier to learn.

2. Object-Oriented

Java uses classes and objects. It supports inheritance, encapsulation, abstraction, and polymorphism. These features help organize and reuse code.

3. Platform-Independent

Java source code is compiled into bytecode. A compatible JVM can execute this bytecode on different operating systems.

4. Secure

Java provides features such as bytecode verification, access control, and runtime checks to help protect programs.

5. Robust

Java supports exception handling, type checking, and automatic memory management. These features help reduce common programming errors.

6. Multithreaded

Java allows multiple threads to perform tasks concurrently. For example, an application may process data while responding to user input.

7. Portable

Java bytecode and standard libraries help programs behave consistently across supported platforms.

8. High-Performance

The JVM uses Just-In-Time (JIT) compilation to convert frequently executed bytecode into machine code, improving execution speed.

9. Distributed

Java provides networking libraries that help programs communicate across computers and networks.

10. Automatic Memory Management

Java's garbage collector can reclaim memory occupied by objects that are no longer reachable.

- S – Simple
- O – Object-Oriented
- P – Platform-Independent
- S – Secure
- R – Robust
- M – Multithreaded
- P – Portable
- H – High-Performance
- D – Distributed
- A – Automatic Memory Management

4. Why is Java platform-independent?

Meaning: Platform independence means that a program can run on different operating systems without needing to rewrite its source code for each one.

Java follows the principle “Write Once, Run Anywhere” (WORA), provided a compatible Java runtime is available.

Flowchart

Java source code

Hello.java

Java compiler (javac)

Converts source code into bytecode

Bytecode

Hello.class

Windows

JVM

Linux

JVM

macOS

JVM

Steps

1. The programmer writes the source code in a .java file.
2. The Java compiler converts the source code into bytecode.
3. The bytecode is stored in a .class file.
4. A JVM on the target platform loads and executes the bytecode.
5. The program produces the required output.

The important point is that the same bytecode can be used on supported platforms, even though each platform needs its own compatible JVM.

5. Simple Java program

```
class Hello {
   public static void main(String[] args) {
      System.out.println("Welcome to Java");
```

```
    }
}
```

Output:

Welcome to Java

This program can be compiled into bytecode and run on different platforms with a compatible Java runtime.

6. Uses of Java

- Banking and financial applications.
- Web applications and server-side software.
- Enterprise applications.
- Desktop applications.
- Networking and distributed systems.
- Educational and data-processing applications.

7. Advantages

- Java is comparatively easy to learn.
- It supports object-oriented programming.
- It provides platform independence through bytecode and the JVM.
- It has automatic memory management.
- It supports exception handling and multithreading.
- It provides many standard libraries for application development.

8. Conclusion

Java is a popular programming language because it is object-oriented, portable, secure, and robust. Its most important feature is platform independence, which is achieved through bytecode and the JVM.

Memory trick: Simple, Object-oriented, Platform-independent, Secure, Robust, Multithreaded, Portable, High-performance, Distributed, Automatic memory management.

2. Explain JVM, JRE, and JDK with a simple diagram

1. Definition

JVM, JRE, and JDK are important components of the Java environment. They help in executing Java programs and developing new Java applications.

2. Meaning

JVM (Java Virtual Machine): It is a virtual machine that loads, verifies, and executes Java bytecode.

JRE (Java Runtime Environment): Traditionally, it refers to the JVM and the libraries and supporting files needed to run Java applications.

JDK (Java Development Kit): It provides the compiler and other development tools used to write, compile, and debug Java programs, along with runtime components.

3. Special features

A. JVM

- Executes Java bytecode.
- Loads Java classes when required.
- Checks bytecode before execution.
- Manages memory during program execution.
- Uses garbage collection to reclaim eligible memory.
- Helps Java programs run on different platforms.

B. JRE

- Provides the runtime environment for Java programs.
- Includes the JVM and required libraries in traditional Java installations.
- Supports program execution without requiring the programmer to compile source code.
- Supplies standard classes and supporting runtime components.

C. JDK

- Contains the Java compiler, javac.
- Provides tools to run, debug, and document programs.
- Supports the development of Java applications.

- Includes the components needed to compile and run programs.

4. Diagram

JDK

Compiler, debugger, development tools

JRE

Runtime libraries and supporting files

JVM

Loads and executes bytecode

Exam note: This is the traditional conceptual relationship. Modern Java distributions may package runtime components differently, and a separate JRE is not always distributed.

5. Working steps

1. Write the program using a text editor or an IDE.
2. Save the program with a .java extension.
3. Use the JDK compiler to convert the source code into bytecode.
4. Store the bytecode in a .class file.
5. The JVM loads and executes the bytecode.
6. The program displays the output.

Example program

```
class Demo {
   public static void main(String[] args) {
      System.out.println("Java is easy");
   }
}
```

Output:

Java is easy

To compile and run the program using the command line:

javac Demo.java

java Demo

6. Uses

- JVM: Executing compiled Java programs.
- JRE: Providing the environment required to run Java applications.
- JDK: Developing, compiling, testing, and debugging Java programs.

7. Advantages

- Each component has a clear purpose.
- The JVM supports cross-platform execution.
- Runtime libraries reduce the need to write everything from scratch.
- Development tools make programming and debugging easier.

8. Conclusion

JVM, JRE, and JDK work together in Java development and execution. The JDK develops programs, the JRE provides runtime support, and the JVM executes bytecode.

Memory trick: Develop → Run → Execute.

3. Explain the structure of a Java program with an example

1. Definition

The structure of a Java program refers to the arrangement of its components, such as comments, class declarations, methods, variables, and statements.

2. Meaning

Every Java program is organized according to Java's syntax rules. In a basic standalone Java application, the main() method is the starting point of execution.

The program is written, compiled, and executed to produce the required result.

3. Main parts of a Java program

1. Comments

Comments explain the purpose of the program. They are ignored by the compiler.

// This is a single-line comment

Java also supports multiline comments:

/*

This is a

multiline comment.

*/

2. Package declaration

A package groups related classes. If used, the package declaration normally appears at the beginning of the source file.

package myprogram;

It is optional for simple programs.

3. Import statement

An import statement allows a program to use classes from other packages without writing their full names each time.

import java.util.Scanner;

It is optional when no additional imported classes are needed.

4. Class declaration

A class is a blueprint used to define objects and organize program members.

class Student {

}

5. Main method

The main method is the standard entry point for a basic Java application launched by the Java launcher.

public static void main(String[] args)

Meaning of each word:

- public: Allows the launcher to access the method.
- static: Allows it to be called without creating an object of the class.

- void: The method does not return a value.
- main: The method name.
- String[] args: Holds command-line arguments.

6. Statements

Statements are instructions that perform actions.

System.out.println("Hello");

7. Braces and semicolons

- { } mark the beginning and end of a class or method block.
- ; marks the end of most Java statements.

4. General syntax

```
// Comments
package packageName;       // Optional
import packageName.ClassName; // Optional


class ClassName {
    public static void main(String[] args) {
        // Statements
    }
}
```

5. Structure diagram

Comments

Package and import statements (optional)

Class declaration

main() method

Variables and executable statements

6. Simple Java program

```
// Program to display student details
class Student {
    public static void main(String[] args) {
        String name = "Ravi";
        int marks = 90;

        System.out.println("Student Name: " + name);
        System.out.println("Marks: " + marks);
    }
}
```

Output:

Student Name: Ravi

Marks: 90

Explanation

- class Student declares the class.
- main() provides the entry point.
- String name stores the student's name.
- int marks stores the marks.
- System.out.println() displays the details.
- The + operator joins text and values in the output.

7. Algorithm / steps

1. Start the program.
2. Declare the class.
3. Write the main() method.
4. Declare the required variables.
5. Assign values to the variables.

6. Display the values using output statements.
7. End the program.

8. Uses and advantages

- Helps organize the program clearly.
- Makes code easier to read and understand.
- Allows programmers to divide a large program into classes and methods.
- Makes testing, debugging, and maintenance easier.
- Supports reuse through classes and methods.

9. Conclusion

A Java program follows a clear structure consisting of a class, methods, variables, and statements. Understanding this structure is the first step toward writing correct Java programs.

Memory trick: Comments → Package/import → Class → Main → Variables → Statements → Output.

4. Explain data types, variables, and operators in Java

1. Definition

Data types, variables, and operators are basic elements of Java programming. Data types define the kind of value, variables store values, and operators perform operations on those values.

2. Meaning

For example, a student's age can be stored as an integer, the student's name as a string, and the marks can be added using the arithmetic + operator.

These three concepts are used in almost every Java program.

3. Data types in Java

A data type tells Java what kind of value a variable can store.

Java data types are divided into two categories.

A. Primitive data types

Java has eight primitive data types.

Data type	Purpose	Example
byte	Small integer values	byte a = 10;
short	Short integer values	short b = 1000;
int	Whole numbers	int age = 18;
long	Large-range integers	long n = 100000L;
float	Decimal values with single precision	float x = 5.5f;
double	Decimal values with double precision	double y = 25.75;
char	One UTF-16 code unit	char grade = 'A';
boolean	true or false	boolean pass = true;

B. Non-primitive / reference types

These store references to objects or arrays.

Examples include:

- String – stores text.
- Arrays – store multiple elements.
- Classes – define objects and their behavior.
- Interfaces – define a contract that classes can implement.

Example:

```
String name = "Ravi";
int[] marks = {80, 90, 85};
```

4. Variables in Java

A variable is a named location used to store a value that a program can use.

Syntax

```
dataType variableName = value;
```

Example:

```
int age = 18;
```

Types of variables by declaration

1. Local variable: Declared inside a method or block. It must be assigned a value before it is used.

2. Instance variable: Declared inside a class but outside methods, without static. Each object has its own instance fields.

3. Static variable: Declared using static. It belongs to the class and is shared by its instances.

Example:

```
class Student {
    int marks = 90;                 // Instance variable
    static String college = "ABC"; // Static variable


    void display() {
        int age = 18;               // Local variable
        System.out.println(age);
    }
}
```

5. Operators in Java

An operator is a symbol that performs an operation on one or more values.

| Type | Operators | Purpose | |---|---|---| | Arithmetic | + - * / % | Calculations | | Relational | == != > < >= <= | Comparisons | | Logical | && || ! | Logical conditions | | Assignment | = += -= *= | Assigning and updating values | | Unary | ++ -- | Increment and decrement | | Conditional | ?: | Selects one of two expressions | | Bitwise | & | ^ ~ | Performs bit-level operations | | Shift | << >> >>> | Shifts bits |

Examples:

```
int a = 10, b = 5;
System.out.println(a + b);  // 15
System.out.println(a > b);  // true
```

```
System.out.println(a == b); // false
```

6. Simple Java program

```
class Basics {
    public static void main(String[] args) {
        int a = 10;
        int b = 5;
        int sum = a + b;

        boolean result = a > b;

        System.out.println("Sum = " + sum);
        System.out.println("A is greater = " + result);
    }
}
```

Output:

```
Sum = 15
A is greater = true
```

7. Algorithm / steps

1. Start the program.
2. Declare two integer variables.
3. Assign values to them.
4. Use the arithmetic operator to calculate the sum.
5. Use the relational operator to compare the values.
6. Display the results.
7. End the program.

8. Uses

- Data types store suitable kinds of data.
- Variables store values used in calculations.
- Arithmetic operators perform mathematical operations.
- Relational and logical operators help make decisions.
- Assignment operators update variable values.

9. Advantages

- Improves the clarity of programs.
- Helps store and process data correctly.
- Makes calculations easier.
- Allows conditions to be checked.
- Supports the development of larger applications.

10. Conclusion

Data types, variables, and operators are essential parts of Java. They help store information, process values, perform calculations, and make decisions.

Memory trick: Data type = Kind, Variable = Storage, Operator = Action.

5. Explain decision-making statements and looping statements in Java

1. Definition

Decision-making statements are used to select a block of code based on a condition.

Looping statements are used to execute a block of code repeatedly while a condition is satisfied or until the required repetitions are completed.

2. Meaning

In real life, we make decisions based on conditions. For example, if a student's marks are 40 or more, the student passes; otherwise, the student fails.

Similarly, if we want to print numbers from 1 to 10, we can use a loop instead of writing ten separate print statements.

3. Decision-making statements

Java provides the following main decision-making statements.

A. if statement

The if statement executes a block only when its condition is true.

Syntax:

```
if (condition) {
    // Statements
}
```

Example:

```
if (age >= 18) {
    System.out.println("Adult");
}
```

B. if-else statement

It selects one of two blocks based on whether a condition is true or false.

Syntax:

```
if (condition) {
    // True block
} else {
    // False block
}
```

C. else-if ladder

It checks multiple conditions in sequence. The first matching block is executed.

Syntax:

```
if (condition1) {
    // Statements
} else if (condition2) {
    // Statements
} else {
```

```
    // Default statements
}
```

It is useful for assigning grades based on marks.

D. Nested if

An if statement written inside another if statement is called a nested if.

It is useful when a second condition should be checked only after the first condition is satisfied.

E. switch statement

A switch statement selects a matching case from several alternatives.

Syntax:

```
switch (choice) {
    case 1:
        System.out.println("One");
        break;
    case 2:
        System.out.println("Two");
        break;
    default:
        System.out.println("Invalid choice");
}
```

The break statement normally exits the switch after a matching case. Without it, execution may continue into subsequent case labels. Modern Java also supports switch expressions and arrow-style cases.

4. Looping statements

Java has three basic loop statements.

A. for loop

The for loop is useful when the number of repetitions is known.

Syntax:

```
for (initialization; condition; update) {
    // Statements
}
```

Working:

1. Initialize the loop variable.
2. Check the condition.
3. Execute the loop body if true.
4. Update the variable.
5. Repeat until the condition becomes false.

B. while loop

The while loop checks its condition before executing the body.

Syntax:

```
while (condition) {
    // Statements
}
```

If the condition is false initially, the body may not execute even once.

C. do-while loop

The do-while loop executes the body first and checks the condition afterward.

Syntax:

```
do {
    // Statements
} while (condition);
```

Therefore, the body executes at least once.

Comparison of loops

Feature	for	while	do-while

Condition checked	Before body	Before body	After body
Minimum executions	Zero	Zero	One
Common use	Known repetitions	Condition-based repetition	Must execute at least once

5. Flowchart

Start

Check condition

True

Execute statements

Update value

Return to condition

False

Exit the loop

End

6. Simple Java program

This program demonstrates both decision-making and looping statements.

```
class DecisionLoop {
    public static void main(String[] args) {
        int marks = 75;

        if (marks >= 40) {
            System.out.println("Pass");
        } else {
            System.out.println("Fail");
        }
```

```
        for (int i = 1; i <= 5; i++) {
            System.out.println(i);
        }
    }
}
```

Output:

```
Pass
1
2
3
4
5
```

Explanation

- marks >= 40 checks the student's result.
- The if-else statement prints Pass.
- The for loop starts at 1.
- It prints each number while i <= 5.
- The i++ statement increases the value by one.

7. Algorithm / steps

1. Start the program.
2. Declare and initialize the marks.
3. Check whether marks are at least 40.
4. Display Pass or Fail.
5. Start a loop with i = 1.
6. Print i while it is at most 5.

7. Increase i by one each time.
8. Stop the loop and end the program.

8. Uses

- Checking student results.
- Creating menu-driven applications.
- Validating user input.
- Printing number sequences.
- Processing arrays.
- Repeating calculations.
- Developing games and business applications.

9. Advantages

- Decision-making statements allow programs to respond to conditions.
- Loops reduce repeated code.
- They make programs shorter and easier to maintain.
- They are useful for processing large amounts of data.
- They support logical problem-solving.

10. Conclusion

Decision-making statements control which block of code is executed. Looping statements repeat code when required. Both are important for writing logical and efficient Java programs.

Memory trick: if = Decide; switch = Select; for = Known repetitions; while = Check first; do-while = Execute first.

6. Explain arrays in Java with examples

1. Definition

An array in Java is an object that stores a fixed number of elements of the same array component type. Each element can be accessed using its index.

2. Meaning

Suppose we want to store the marks of five students. We could create five separate variables, but this becomes difficult when there are many students.

An array allows us to store the marks under one variable name and access each mark using an index.

3. Special features of arrays

1. Fixed length: The length of an array is set when the array is created.
2. Same component type: An int[] stores integer values; a String[] stores string references.
3. Index-based access: Elements are accessed by their positions.
4. Zero-based indexing: The first element has index 0.
5. Direct access: An element can be accessed by index in constant time.
6. Length property: The number of elements can be obtained using arrayName.length.
7. Arrays are objects: They are created using array creation syntax and can be assigned to reference variables.

4. Types of arrays

A. One-dimensional array

A one-dimensional array stores values in a single sequence.

Example:

int[] marks = {80, 90, 75, 85, 95};

Diagram:

Array elements

80

Index 0

90

Index 1

75

Index 2

85

Index 3

95

Index 4

The first element is stored at index 0, and the last is at index 4.

B. Two-dimensional array

A two-dimensional array stores elements in rows and columns. It is commonly used to represent tables and matrices.

Example:

```
int[][] matrix = {
    {1, 2},
    {3, 4}
};
```

Diagram:

2 × 2 Matrix

1

2

3

4

Two rows and two columns

Java also supports multidimensional arrays whose rows can have different lengths.

5. Syntax of arrays

Declaration

```
dataType[] arrayName;
```

Creation

```
arrayName = new dataType[size];
```

Declaration and initialization

```
int[] numbers = {10, 20, 30};
```

Accessing an element

```
System.out.println(numbers[0]);
```

Output:

```
10
```

6. Simple Java program

This program displays array elements and calculates their sum.

```
class ArrayExample {
    public static void main(String[] args) {
        int[] marks = {80, 90, 70};
        int sum = 0;

        for (int i = 0; i < marks.length; i++) {
            System.out.println(marks[i]);
            sum = sum + marks[i];
        }

        System.out.println("Total = " + sum);
    }
}
```

Output:

```
80
90
70
Total = 240
```

Explanation

- int[] marks declares an integer array.
- {80, 90, 70} initializes three elements.
- marks.length gives the number of elements, which is 3.
- marks[i] accesses each element.
- sum = sum + marks[i] adds each mark to the total.
- The last statement displays the total.

7. Algorithm / steps

1. Start the program.
2. Declare an array and initialize its values.
3. Set sum = 0.
4. Set the loop index i = 0.
5. Check whether i is less than the array length.
6. Display the current element.
7. Add the current element to sum.
8. Increase i by one and repeat.
9. Display the total.
10. Stop.

8. Uses of arrays

- Storing student marks.
- Storing employee salaries.
- Representing matrices.
- Sorting and searching values.
- Storing lists of items.
- Processing numerical data.
- Implementing other data structures.

9. Advantages

- Stores multiple values under one name.
- Makes data processing easier.
- Works efficiently with loops.
- Provides direct access through indexes.
- Reduces the need to create many separate variables.

10. Limitations

- The length of an array cannot be changed after creation.
- Inserting or deleting elements in the middle may require shifting other elements.
- Accessing an invalid index causes an ArrayIndexOutOfBoundsException.
- An array of primitive type stores primitive values, while an array of reference type stores object references.

11. Conclusion

Arrays are useful when a program needs to store and process multiple values of the same component type. They make searching, sorting, and calculations easier.

Memory trick: Declare → Initialize → Access → Process → Display.

7. Differentiate between Java and C++

1. Definition

Java: Java is a high-level, class-based, object-oriented programming language that uses bytecode and the JVM to support platform-independent execution.

C++: C++ is a general-purpose programming language that supports procedural programming, object-oriented programming, and other programming techniques.

2. Meaning

Both Java and C++ are used to develop software applications. However, they differ in compilation, memory management, pointers, inheritance, and program execution.

Java focuses on portability and automatic memory management, while C++ provides more direct control over memory and system resources.

3. Differences between Java and C++

Feature	Java	C++
Platform independence	Bytecode runs on compatible JVMs	Programs are commonly compiled for a particular platform
Programming approach	Primarily class-based and object-oriented	Procedural, object-oriented, and other approaches
Compilation	Compiles source code into bytecode	Commonly compiles source code into native machine code
Memory management	Automatic garbage collection	Manual management and other memory-management techniques
Pointers	No direct pointer manipulation	Supports pointers
Multiple inheritance	A class cannot extend multiple classes	A class can inherit from multiple classes
Operator overloading	Generally not supported for user-defined classes	Supported
Header files	Does not use C++-style header files	Commonly uses header files
Preprocessor	Does not use a C++-style preprocessor	Supports preprocessor directives
Templates / generics	Uses generics	Uses templates
Destructor	No deterministic class destructors; uses garbage collection and other resource-management methods	Supports destructors
Common applications	Enterprise software, server applications, many Android applications	Games, system software, embedded software, high-performance applications

4. Special features of Java

1. Bytecode supports execution on different platforms.
2. Garbage collection helps manage memory automatically.
3. Classes and interfaces support object-oriented design.
4. Exception handling helps manage runtime errors.
5. Built-in libraries support networking, collections, and input/output.

5. Special features of C++

1. Supports both procedural and object-oriented programming.
2. Provides pointers and low-level memory access.
3. Supports operator overloading.
4. Supports multiple inheritance of classes.
5. Offers detailed control over resources and performance.
6. Supports templates for generic programming.

6. Simple program examples

Java program

```
class Hello {
    public static void main(String[] args) {
        System.out.println("Hello");
    }
}
```

C++ program

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello";
    return 0;
```

}

Output of both programs:

Hello

Explanation

- In Java, System.out.println() displays output.
- In C++, cout displays output.
- Both programs print the same message, but their syntax and execution environments are different.

7. Uses

Java is used for:

- Enterprise applications.
- Web backends and server software.
- Banking systems.
- Networking applications.
- Many Android applications.

C++ is used for:

- Game engines.
- Operating-system components.
- Embedded systems.
- High-performance applications.
- Graphics and system software.

8. Advantages

Advantages of Java

- Platform independence through bytecode and the JVM.
- Automatic garbage collection.
- Extensive standard libraries.
- Built-in exception-handling support.

Advantages of C++

- Greater low-level control.
- Supports pointers and manual resource management.
- Useful for performance-sensitive applications.
- Supports templates and operator overloading.

9. Conclusion

Java and C++ are both powerful programming languages. Java is commonly chosen when portability and automatic memory management are important, while C++ is often chosen when low-level control and performance are major requirements.

Memory trick: Java = Bytecode + JVM + Garbage collection. C++ = Native code + Pointers + Greater low-level control.

8. Explain type casting in Java

1. Definition

Type casting is the process of converting a value from one data type to another compatible data type.

For example, an integer value can be converted to a double, or a decimal value can be converted to an integer.

2. Meaning

Different operations may require different data types. Type casting helps convert values into the required type.

For example, if a program stores a value as double but needs an integer result, it can use an explicit cast.

Java type conversion is commonly explained using two main types: widening and narrowing.

3. Types of type casting

A. Widening casting (implicit casting)

Widening converts a value from a smaller-range numeric type to a larger-range numeric type. It normally happens automatically.

Common conversions include:

byte

short → int → long → float → double

Simplified view of common widening conversions

A widening conversion generally does not require explicit casting. However, some conversions, such as int to float, may lose precision for certain values.

Syntax:

largerType variable = smallerTypeValue;

Example:

int a = 10;

double b = a;

Here, the value of a is converted to 10.0.

B. Narrowing casting (explicit casting)

Narrowing converts a value into a type that has a smaller range or less precision. It usually requires the programmer to specify the target type.

Syntax:

targetType variable = (targetType) value;

Example:

double x = 15.75;

int y = (int) x;

Here, y becomes 15. The fractional part is discarded; the value is not rounded to the nearest integer.

4. Special features of type casting

- Converts compatible values from one type to another.
- Widening conversions are generally automatic.
- Narrowing conversions usually require an explicit cast.
- Numeric narrowing can lose information.
- Type casting can be used in arithmetic expressions.

- Casting does not always change the original variable; it produces a converted value.

5. Simple Java program

```
class TypeCasting {
    public static void main(String[] args) {
        int a = 10;
        double b = a;

        double x = 15.75;
        int y = (int) x;

        System.out.println("Widening: " + b);
        System.out.println("Narrowing: " + y);
    }
}
```

Output:

Widening: 10.0

Narrowing: 15

6. Explanation of the program

1. int a = 10 stores an integer value.
2. double b = a converts the integer into a double automatically.
3. double x = 15.75 stores a decimal value.
4. (int) x converts the decimal into an integer.
5. The decimal portion .75 is discarded.
6. The println() statements display both converted values.

7. Algorithm / steps

1. Start the program.

2. Declare an integer variable and assign a value.
3. Convert it into a double using widening casting.
4. Declare a double variable with a decimal value.
5. Convert it into an integer using explicit narrowing casting.
6. Display both results.
7. Stop.

8. Uses of type casting

- Converting numeric values for calculations.
- Working with different numeric data types.
- Converting decimal values into integers when required.
- Passing compatible values to methods that expect a particular type.
- Processing data obtained from different sources.

9. Advantages

- Allows compatible data types to work together.
- Makes data conversion possible when required.
- Supports calculations involving different numeric types.
- Provides control over the target type of a value.

10. Disadvantages

- Narrowing may remove decimal information.
- Values outside the target type's range may produce unexpected results.
- Incorrect casts can cause errors.
- Casting between incompatible reference types can cause a ClassCastException at runtime.

11. Conclusion

Type casting is an important feature of Java that converts values from one data type to another. Widening casting is generally automatic, while narrowing casting usually requires an explicit cast.

Memory trick: Widening = automatic conversion; Narrowing = explicit conversion.